WhatsApp Home Orchestrator

A WhatsApp assistant running in AWS. Messages sent to a WhatsApp Business number reach a serverless backend, where Claude (Anthropic's model, on Amazon Bedrock) decides what to do. It can control SmartThings devices, set reminders, keep lists, read voice notes and photos, and look things up. It then replies on WhatsApp.

2 Lambda functions
Node.js 22, arm64, TypeScript

Claude Haiku 4.5
via Amazon Bedrock

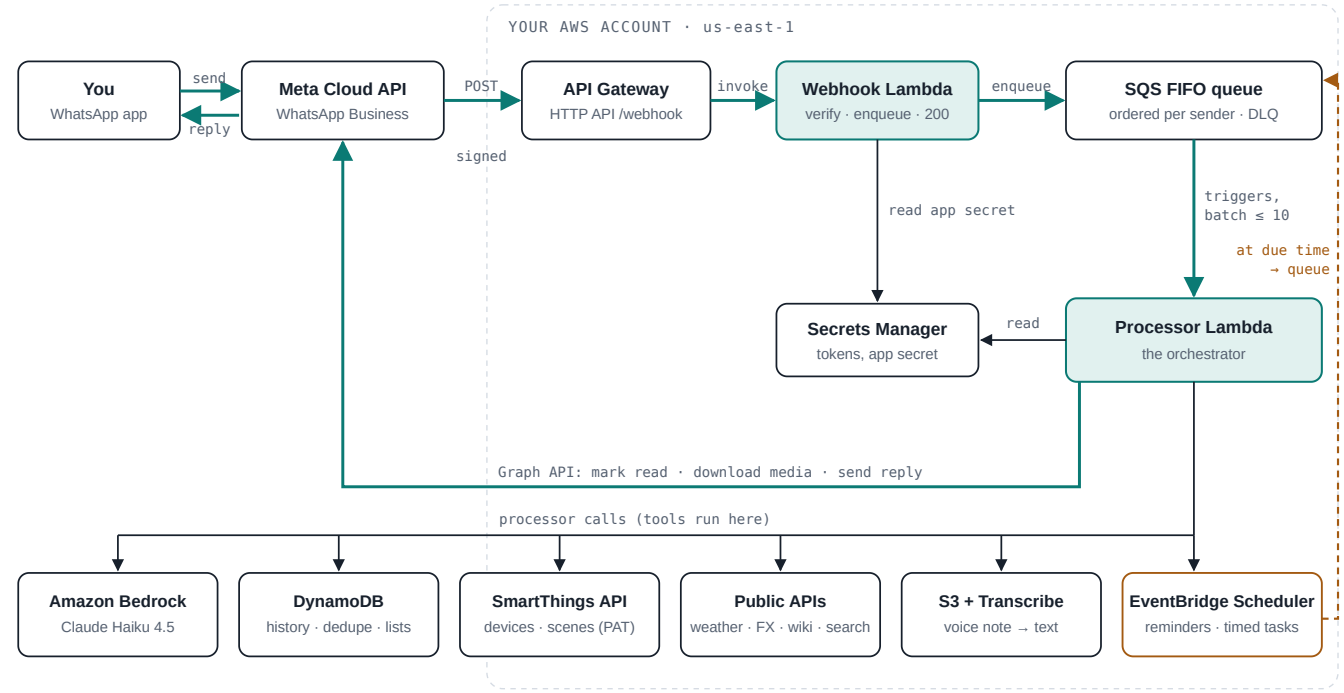
~20 tools
in 4 plug-in providers

1 SAM template
creates every AWS resource

1. Architecture at a glance
2. AWS services used
3. Message flows
4. The AI orchestrator
5. Features
6. What the platform takes care of
7. The code deployed
8. Configuration and secrets
9. Data stored
10. Permissions (IAM)
11. Operating it
12. Cost
13. Limits and roadmap
14. Worked example, step by step
15. Design decisions and why
16. How the project was set up
17. Glossary

1. Architecture at a glance

The design splits into a fast path that accepts a message and a slow path that does the work. Meta requires the webhook to answer within a few seconds, so the webhook Lambda only verifies and queues. The processor Lambda, triggered from the queue, does everything that takes time: downloading media, calling Claude, running tools and sending the reply.



a message's main path, in and out calls made along the way scheduled items re-entering the queue

Meta reaches AWS only through API Gateway; every reply leaves through Meta's Graph API. Only the processor talks to Claude and the tools. CloudWatch Logs (not drawn) collects JSON logs from both Lambdas.

2. AWS services used

Everything below is declared in `template.yaml` and created by one `sam deploy`. Meta, SmartThings and the public APIs are external services called over HTTPS.

SERVICE	WHAT IT DOES HERE	HOW IT'S CONFIGURED
Amazon API Gateway HTTP API	The public HTTPS URL Meta calls. `GET /webhook` for the one-time verification handshake, `POST /webhook` for message events.	Two routes into the webhook Lambda. No auth at the gateway (Meta can't send AWS credentials); the Lambda verifies Meta's signature.
AWS Lambda: WebhookFunction	Verifies the request is from Meta, extracts messages, puts them on the queue, answers 200 immediately.	Node.js 22, arm64, 512 MB, 10 s timeout.
Amazon SQS (FIFO) + DLQ	Buffer between receiving and processing. Keeps each sender's messages in order, drops duplicates, retries failures, parks poison messages.	FIFO, content-based dedup on, visibility timeout 1080 s, redrive to a FIFO dead-letter queue after 3 attempts (DLQ keeps 14 days).
AWS Lambda: ProcessorFunction	The orchestrator: gatekeeping, media handling, the Claude tool loop, the reply.	Node.js 22, arm64, 512 MB, 180 s timeout. SQS trigger, batch size 10, partial batch failure reporting.
Amazon Bedrock	Runs Claude Haiku 4.5, which reads the conversation and decides which tools to call.	Model `us.anthropic.claude-haiku-4-5-20251001-v1:0` (US cross-region inference profile), called with the Lambda's IAM role.

SERVICE	WHAT IT DOES HERE	HOW IT'S CONFIGURED
Amazon DynamoDB	Conversation history, message dedupe markers, users' lists.	One table, on-demand billing, partition key `pk`, TTL on `expiresAt`.
AWS Secrets Manager	Holds every credential as one JSON secret.	Read by both Lambdas and cached for 5 minutes, so rotated tokens apply without a redeploy.
Amazon EventBridge Scheduler	One-time schedules for reminders and timed tasks. At the due time it puts a message on the SQS queue.	Schedule group `sam-app-reminders`; its own IAM role that can only send to the queue; schedules delete themselves after running.
Amazon S3	Temporary storage for voice notes while they are transcribed.	Private bucket (all public access blocked); objects deleted after use, with a 1-day expiry rule as a backstop.
Amazon Transcribe	Turns voice notes into text.	Batch jobs with automatic language detection among English (India), Hindi and English (US). Needs Transcribe enabled on the account.
AWS IAM	Least-privilege roles for each Lambda and for Scheduler.	Generated by SAM from the template; see section 10.
Amazon CloudWatch Logs	Structured JSON logs from both Lambdas, including every tool call.	Read with `sam logs --stack-name sam-app --tail`.
AWS CloudFormation / SAM	Creates and updates the whole stack as one unit.	Stack `sam-app`; settings saved in `samconfig.toml`.

External services

SERVICE	USED FOR	AUTHENTICATION
Meta WhatsApp Cloud API	Receiving messages (webhook), sending replies, read receipts and typing indicator, downloading media.	Webhooks signed with the app secret (HMAC-SHA256). Outbound calls use a system-user access token that never expires.
Samsung SmartThings API	Listing devices and scenes, reading status, sending commands.	Personal access token (expires about every 24 hours; OAuth planned).
Open-Meteo	Geocoding and weather forecasts.	None (free, no key).
Frankfurter	Daily reference exchange rates.	None.
Wikipedia REST API	Article search and summaries.	None.
Tavily	Web search (optional).	API key in the secret; the tool reports "not configured" without it.

3. Message flows

3.1 A text message, end to end

01 **User sends a message** `WhatsApp → Meta`
It goes to the business number, which Meta hosts.

02 **Meta calls the webhook** `Meta → API Gateway → WebhookFunction`
HTTPS POST with a JSON body and an `X-Hub-Signature-256` header (HMAC-SHA256 of the raw body, keyed with the app secret). Delivery receipts (sent, delivered, read) arrive the same way and are ignored.

03 **Signature check** `WebhookFunction → Secrets Manager`
The Lambda recomputes the HMAC and compares it in constant time. Mismatch returns 401 and nothing is queued.

04 **Enqueue and acknowledge** `WebhookFunction → SQS`
Each message becomes a queue entry: group id = sender's number (ordering), dedup id = WhatsApp message id (drops Meta's retries). Returns 200 to Meta at once.

05 **Processor is triggered** `SQS → ProcessorFunction`
Up to 10 records per invocation, handled sequentially. After a failure the rest of the batch is returned too, so order is kept.

06 **Gatekeeping** `ProcessorFunction → DynamoDB`
Ignore other business numbers and senders not on the allow list. Claim the message id with a conditional write; if it already exists, skip it.

07 **Read receipt** `ProcessorFunction → Graph API`
Blue ticks plus the typing indicator.

08 **Orchestration** `ProcessorFunction ⇄ Bedrock ⇄ tools`
Load the last 10 turns, call Claude with the system prompt, current time, history and tool list, run requested tools, repeat until Claude answers (section 4).

09 **Save and reply** `ProcessorFunction → DynamoDB, Graph API`
Save the turn to history (24 h), send the reply (split above 4,000 characters). A failed send is logged and never blocks the queue.

3.2 Other flows

Webhook verification (once)

1. In the Meta dashboard you enter the callback URL and a verify token.
2. Meta sends `GET /webhook?hub.mode=subscribe&hub.verify_token=…&hub.challenge=…`.
3. The Lambda compares the token with the secret and echoes the challenge.

Voice note

1. The webhook queues the audio media id.
2. The processor gets a download URL from the Graph API and downloads the OGG/Opus file.
3. It uploads to S3 and starts a Transcribe job with language detection.
4. It polls every 1.5 s (up to 90 s), reads the transcript, deletes the file and job.
5. The transcript goes to Claude; the reply starts with 🎤 and what was heard.

Photo

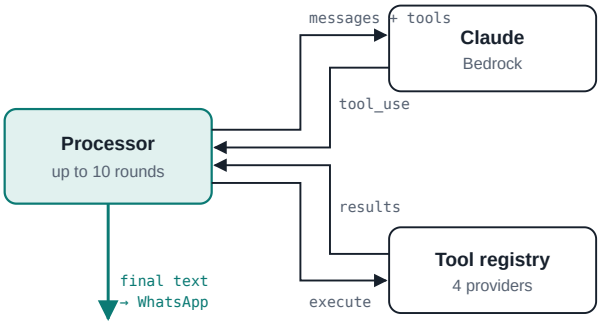
1. The webhook queues the image id and caption.
2. The processor downloads the image from Meta.
3. It sends Claude a base64 image block plus the caption; Claude reads it directly.
4. Only "[photo] caption" is saved to history.

Reminder and timed task

1. Claude calls `set_reminder` or `schedule_task`.
2. The processor creates a one-time EventBridge schedule named after the sender, in UTC.
3. At the due time Scheduler puts a message on the same FIFO queue.
4. Reminder: sends "⏰ Reminder: …". Task: runs the full orchestrator with the saved instruction (e.g. turn off the AC).

4. The AI orchestrator

The processor does not hard-code what to do with a message. Claude reads the request and chooses tools; the code runs them and returns results until Claude produces a final answer. Claude never touches devices or AWS directly: it can only ask, and the code decides what runs.



Up to 10 rounds per message. Calls in one round run in parallel. A failing tool returns an error to Claude instead of crashing, so Claude can explain or try another way.

Each request to Claude contains

- **System prompt:** general-assistant role, WhatsApp reply style, and each provider's usage rules (e.g. confirm before unlocking).
- **Current date and time** in the user's zone (default Asia/Kolkata), so "tomorrow at 7" works.
- **History:** the last 10 user/assistant turns from DynamoDB.
- **The new message:** text, a transcript, or an image plus caption.
- **Tool definitions:** name, description and JSON schema for every tool.

Client selection

Versioned model ids and inference profiles (`us.anthropic…`) use the Bedrock Runtime client; short ids use the Bedrock Mantle Messages endpoint. The effort setting is skipped for Haiku, which doesn't accept it. Both are handled in `orchestrator.ts`.

Tool registry

Tools come from providers that implement one `ToolProvider` interface (`id`, `instructions`, `tools`, `execute`). The registry routes each call to its owner and rejects duplicate tool names. Adding an integration means writing one provider and registering it; the orchestrator is unchanged.

5. Features

AREA	TOOLS	WHAT THE USER CAN DO
Smart home SmartThings	`list_devices`, `get_device_status`, `send_device_command`, `list_scenes`, `execute_scene`	"Turn off the bedroom light", "is the door locked?", "set the AC to 24", "run Good Night". Filters by room or capability; never invents device ids; asks before unlocking, opening a garage or disarming.
Reminders EventBridge Scheduler	`set_reminder`, `schedule_task`, `list_reminders`, `cancel_reminder`	"Remind me at 6 pm to call mum", "turn off the AC in 2 hours", "what's scheduled?", "cancel it". Up to a year ahead; security actions refused on a timer.
Lists and notes DynamoDB	`add_to_list`, `remove_from_list`, `get_list`, `show_lists`, `delete_list`	"Add milk and eggs to my shopping list", "what's on my to-do list?", "note the wifi password is on the fridge". Case-insensitive, no duplicates, kept indefinitely.

AREA	TOOLS	WHAT THE USER CAN DO
Live information	`get_current_time`, `get_weather`, `convert_currency`, `wikipedia_lookup`, `web_search`	Time in any zone, current weather and up to 7-day forecast, currency conversion, encyclopedia facts, and web search for news once a Tavily key is added.
Voice notes	(input handling)	Speak instead of typing, in English or Hindi. The reply shows what was heard.
Photos	(input handling)	Send a photo, optionally with a question; Claude describes or answers about it.
Conversation	(no tool)	General questions, explanations, drafting, translation. Remembers the last 10 turns for 24 hours.

6. What the platform takes care of

Security

- **Authenticity:** every POST must carry a valid HMAC from Meta's app secret; forged requests get 401.
- **Access control:** only numbers in `AllowedSenders` are served; if the list is empty, nobody is served.
- **Risky actions:** confirmation required before unlocking doors, opening garages or gates, or disarming security; never scheduled.
- **Secrets:** all tokens in Secrets Manager, none in code or template.
- **Least privilege:** each role reaches only this stack's table, secret, queue, bucket and schedule group.
- **Scoped reminders:** users can list or cancel only schedules named after their own number.
- **Media hygiene:** voice notes are deleted after transcription; the bucket is private with a 1-day expiry.

Reliability

- **Fast acknowledgement:** the webhook only verifies and queues, so Meta never times out.
- **Ordering:** FIFO groups by sender, so one person's messages run in sequence.
- **Exactly-once effect:** SQS dedup (5 min) plus a DynamoDB claim per message id; a retry never repeats a command.
- **No head-of-line blocking:** a failed reply is logged instead of retried, so later messages keep flowing.
- **Retries and DLQ:** processing failures retry up to 3 times, then move to the dead-letter queue.
- **Graceful tool errors:** a failing tool returns an error to Claude, which tells the user in plain words.
- **Loop guard:** at most 10 tool rounds per message.
- **Timeouts:** every outbound HTTP call has a timeout (10–30 s).
- **Token rotation:** secrets re-read every 5 minutes, no redeploy.

Observability

Both Lambdas log single-line JSON: each webhook request (method, source IP, user agent, body size), queued count, skipped senders, duplicates, every tool call with provider, input and errors, transcription results and send failures. Lambda platform reports add duration and memory per invocation.

7. The code deployed

TypeScript, bundled by esbuild into one file per Lambda (`dist/webhook`, `dist/processor`). SAM uploads those bundles; no `sam build` step. Libraries: `@anthropic-ai/bedrock-sdk` and AWS SDK v3 clients for DynamoDB, SQS, Secrets Manager, Scheduler, S3 and Transcribe.

FILE	RESPONSIBILITY
src/handlers/webhook.ts	GET verification, POST signature check, message extraction (text, captions, buttons, audio and image ids), SQS batch send.
src/handlers/processor.ts	SQS consumer: number and allow-list filters, dedupe claim, read receipt, voice and photo input, scheduled items, orchestrator call, history, safe reply.
src/orchestrator/orchestrator.ts	System prompt, Bedrock client choice, the tool-use loop, parallel tool execution, stop reasons.
src/orchestrator/registry.ts, types.ts	`ToolProvider` interface and the registry that routes tool calls.
src/general/provider.ts	Time, weather, currency, Wikipedia and web search tools.
src/reminders/provider.ts	Reminder and timed-task tools on EventBridge Scheduler, time-zone conversion.
src/lists/provider.ts	List and note tools on DynamoDB.
src/smartthings/provider.ts, client.ts, auth.ts	SmartThings tools, typed REST client, token provider (PAT now; OAuth slot).
src/whatsapp/client.ts, media.ts, signature.ts, types.ts	Graph API send and mark-read, media download, HMAC check, webhook types.
src/media/transcribe.ts	S3 staging and Amazon Transcribe job handling.
src/store/conversation.ts	History and message-dedupe items in DynamoDB.
src/config.ts	Environment settings and the cached secret loader.
src/local/chat.ts	Terminal chat for testing the orchestrator without WhatsApp.
template.yaml, scripts/build.mjs	The AWS stack definition and the esbuild bundling script.

8. Configuration and secrets

Deploy parameters (asked by `sam deploy --guided`)

PARAMETER	MEANING	CURRENT VALUE
WhatsAppPhoneNumberId	Which business number this stack serves	Meta test number's id
AllowedSenders	Comma-separated numbers allowed to use the bot	Owner's number
BedrockModelId	Claude model or inference profile	`us.anthropic.claude-haiku-4-5-20251001-v1:0`
ModelEffort	Reasoning effort for models that support it	medium (ignored for Haiku)
WhatsAppGraphApiVersion	Graph API version	v23.0

Secret (one JSON value in Secrets Manager)

KEY	PURPOSE
WHATSAPP_VERIFY_TOKEN	Shared string for Meta's webhook verification
WHATSAPP_APP_SECRET	Verifies webhook signatures
WHATSAPP_ACCESS_TOKEN	System-user token for sending, read receipts and media download
SMARTTHINGS_PAT	SmartThings personal access token (devices list, see, control; scenes)
TAVILY_API_KEY `optional`	Turns on web search

Other environment variables (table name, queue URL and ARN, schedule group, scheduler role, media bucket, allowed senders) are set by the template and need no manual setup.

9. Data stored

DYNAMODB ITEM	CONTENTS	LIFETIME
`CONV#<number>`	Last 10 user/assistant turns, text only (photos stored as "[photo] caption")	24 h after last message
`MSG#<message id>`	Marker that a message or scheduled item was handled	3 days
`LISTS#<number>`	The user's lists: name → items	Until deleted

Reminders live as EventBridge schedules (deleted after firing). Voice notes exist in S3 only during transcription. No photos or audio are kept.

10. Permissions (IAM)

ROLE	ALLOWED ACTIONS
WebhookFunction	Send messages to this queue; read this secret.
ProcessorFunction	Read/write this DynamoDB table; read this secret; invoke Bedrock models; create, read and delete schedules in this group, list schedules; pass the scheduler role; start, read and delete Transcribe jobs; read/write this media bucket; consume the queue.
SchedulerRole	Assumed only by EventBridge Scheduler in this account; can only send messages to this queue.

11. Operating it

```
npm install            # once, or after dependencies change
npm run build          # bundle both Lambdas with esbuild
sam deploy             # update the stack (--guided the first time)
sam logs --stack-name sam-app --tail          # live logs
sam logs --stack-name sam-app --start-time "20min ago" --filter "tool_call"
```

SYMPTOM	LIKELY CAUSE AND FIX
No reply, log shows `WhatsApp Graph API 401`	Access token expired or wrong; put a valid system-user token in the secret, wait 5 minutes.
Nothing in logs at all	Meta isn't calling the webhook: check the callback URL, the **messages** field subscription, and that the WABA is subscribed to the app.
`ignored sender not on allow list`	Number format differs; use country code, no +.
SmartThings 401	PAT expired or missing scope; generate a new one with device control scope.
`SubscriptionRequiredException` on voice notes	Amazon Transcribe not enabled on the account; upgrade from the Free plan or wait for activation.
Reminder never arrives	Outside WhatsApp's 24-hour window, or the schedule failed; check `send reply failed` in logs.

12. Cost

Estimate for about 10 requests a day (300 a month), us-east-1 list prices:

ITEM	MONTHLY
Bedrock, Claude Haiku 4.5	about $1.50–3.00 (most of the bill)
Secrets Manager (1 secret)	about $0.40
Lambda, API Gateway, SQS, DynamoDB, Scheduler, S3	free tier, or a few cents
CloudWatch Logs	a few cents
Amazon Transcribe	about $0.024 per minute of audio (60 free minutes a month in the first year)
WhatsApp replies to user messages, SmartThings, public APIs	free
Total	**roughly $2–4 a month**; nothing is charged while idle

13. Limits and roadmap

Known limits. WhatsApp allows free-form business messages only within 24 hours of the user's last message, so far-off reminders need the user to have chatted recently. The SmartThings PAT expires about daily. The bot still uses Meta's test number. Voice notes require Transcribe to be enabled.

- **SmartThings OAuth** with refresh tokens, per user, replacing the PAT (the `SmartThingsTokenProvider` interface is ready for it).
- **Own business number** instead of Meta's test number.
- **Approved message templates** so reminders can arrive outside the 24-hour window.
- **Tighter IAM** on Bedrock (specific model ARNs) and alarms on the dead-letter queue.
- **More providers** (calendar, email, other smart-home platforms) through the same plug-in interface.

14. Worked example, step by step

The user sends: *"Turn off the bedroom light and remind me in 30 minutes to lock the front door."* This is what each component does, in order.

14.1 What Meta sends to the webhook (trimmed)

```
POST /webhook
X-Hub-Signature-256: sha256=9f2c…   (HMAC of the body with the app secret)

{ "object": "whatsapp_business_account",
  "entry": [{ "changes": [{ "field": "messages", "value": {
      "metadata": { "phone_number_id": "1174…253" },
      "contacts": [{ "wa_id": "9172…354", "profile": { "name": "Ayush" } }],
      "messages": [{ "id": "wamid.HBgM…", "from": "9172…354", "type": "text",
                     "text": { "body": "Turn off the bedroom light and remind me…" } }]
  }}]}]}
```

14.2 What the webhook puts on the queue

```
MessageGroupId:         9172…354          (keeps this sender's messages in order)
MessageDeduplicationId: wamid.HBgM…       (drops Meta's retries)
Body: { "messageId": "wamid.HBgM…", "from": "9172…354", "senderName": "Ayush",
        "phoneNumberId": "1174…253", "text": "Turn off the bedroom light and remind me…",
        "messageType": "text" }
```

14.3 The orchestrator rounds

ROUND	CLAUDE ASKS FOR	THE PROCESSOR DOES AND RETURNS
1	`list_devices` with `room: "Bedroom"`, and in parallel `set_reminder` with `message: "Lock the front door"`, `in_minutes: 30`	Calls SmartThings `GET /devices`, filters to the bedroom, returns ids, names and capabilities. Creates an EventBridge schedule `9172…354-mui21…` at now + 30 min (UTC) and returns its id and local time.
2	`send_device_command` for the bedroom light's id: `switch/off`	Calls SmartThings `POST /devices/{id}/commands`; returns success.
3	No tools: a final answer	"Done, the bedroom light is off. I'll remind you to lock the front door at 7:52 pm."

Each tool call is logged as a `tool_call` line with its provider, input and any error. The turn is saved to history and the reply is sent through the Graph API.

14.4 Thirty minutes later

01 **Scheduler fires** `EventBridge Scheduler → SQS`
Using its own role, it sends the saved payload to the FIFO queue with the user's number as the group id: `{"messageId":"sched-9172…354-mui21…","from":"9172…354","text":"Lock the front door","scheduled":{"type":"reminder",…}}`. The schedule then deletes itself.

02 **Processor handles it** `SQS → ProcessorFunction`
Same allow-list and dedupe checks. Seeing `scheduled.type = "reminder"`, it skips Claude and sends "⏰ Reminder: Lock the front door".

Had the user said "lock the front door in 30 minutes" instead, it would be a `schedule_task`. When it fires, the processor runs the orchestrator with that instruction, and Claude would call `send_device_command` with `lock/lock`. Locking is allowed on a timer; unlocking is not.

15. Design decisions and why

DECISION	WHY	ALTERNATIVE CONSIDERED
Two Lambdas with a queue between them	Meta needs a fast 200. AI calls and tool calls can take seconds to minutes. The queue absorbs bursts and gives retries for free.	One Lambda doing everything: risks Meta timeouts, duplicate deliveries and disabled webhooks.
SQS FIFO, grouped by sender	A person's messages must run in order ("turn on the light", then "no, turn it off"). Different people still run in parallel.	Standard SQS: cheaper, but can reorder and duplicate.
Claim each message in DynamoDB	SQS dedup only covers 5 minutes and one queue; the claim guarantees a device command never runs twice on redelivery.	Relying on SQS dedup alone.
Claude chooses tools (orchestrator)	Handles free-form language, combined requests and follow-ups without writing intent rules. New capabilities are just new tools.	Keyword or intent matching: brittle and needs code for every phrasing.
Plug-in `ToolProvider` interface	SmartThings, reminders, lists and general tools are independent modules; adding one doesn't touch the orchestrator.	One big switch statement.
Amazon Bedrock, not a direct model API	IAM authentication (no extra API key), AWS billing, data stays in AWS, same region as everything else.	Anthropic API directly: another key to manage and a separate bill.
Claude Haiku 4.5	Fast and inexpensive, good at tool use, reads images. Larger models were not enabled on the account. Changing model is one deploy parameter.	Opus or Sonnet: smarter but slower and pricier.
EventBridge Scheduler for reminders	Exact one-time schedules, no polling, deletes itself, delivers into the existing queue so reminders get the same ordering and dedupe.	A cron Lambda scanning DynamoDB every minute: more code and constant cost.
Secrets Manager with a 5-minute cache	Tokens rotate without redeploying; caching keeps calls and cost low.	Environment variables: need a redeploy to change and are visible in the console.
Serverless (Lambda, on-demand DynamoDB)	No servers to patch, scales to zero, costs nothing while idle.	An EC2 or container service: always-on cost and maintenance.
AWS SAM, arm64, esbuild	One template for the whole stack; arm64 is cheaper per millisecond; esbuild makes small, fast-starting bundles.	Manual console setup: not repeatable.

16. How the project was set up

01 **Tools installed locally**
Node.js 22, AWS CLI v2 and AWS SAM CLI on Windows; `aws configure` with an IAM user's access key, region us-east-1.

02 **Model access in Bedrock**
Anthropic models enabled in us-east-1. Opus was denied for the account, Claude Haiku 4.5 worked and was chosen.

03 **Meta app**
A Business app with the WhatsApp product; phone number id, app secret and access token collected; the recipient number verified on the test number.

04 **First deploy**
`npm install`, `npm run build`, `sam deploy --guided`. This created the stack and printed the webhook URL.

05 **Secrets filled**
Verify token, app secret, access token and SmartThings PAT saved in the stack's secret.

06 **Webhook connected**
Callback URL and verify token entered in Meta, **messages** field subscribed, app published, and the WhatsApp Business Account subscribed to the app through `POST /{WABA_ID}/subscribed_apps`.

07 **Hardening**
The temporary token was replaced by a never-expiring system-user token; the PAT was regenerated with device-control scope.

08 **Features added**
General tools, reminders and timed tasks, lists, voice notes and photos, each followed by `npm install`, `npm run build`, `sam deploy`.

17. Glossary

TERM	MEANING IN THIS PROJECT
Webhook	A URL that Meta calls whenever something happens on the WhatsApp number.
WABA	WhatsApp Business Account, which owns the phone number and must be subscribed to the Meta app.
Graph API	Meta's HTTPS API used to send messages, mark them read and download media.
HMAC signature	A hash of the request body made with a shared secret, proving the request came from Meta.
FIFO queue	An SQS queue that delivers messages in the order sent within a group and removes duplicates.
Dead-letter queue (DLQ)	Where messages go after failing 3 times, so they can be inspected instead of retried forever.

TERM	MEANING IN THIS PROJECT
Idempotency	Handling the same message twice has the same effect as handling it once.
LLM	Large language model; here Claude Haiku 4.5.
Tool use	The model replying with a structured request to run a named function with given inputs, instead of text.
Orchestrator	The loop that sends the conversation to Claude, runs the tools it asks for and returns the results.
Inference profile	A Bedrock model id (`us.…`) that lets AWS serve requests from any US region with capacity.
Token (LLM)	A unit of text, roughly ¾ of a word; Bedrock bills per token read and written.
PAT	Personal access token for SmartThings, tied to one account.
SAM	AWS Serverless Application Model: a template format and CLI that deploys the stack.
IAM role	A set of AWS permissions a service assumes; each Lambda and Scheduler has its own.
24-hour window	WhatsApp's rule that a business can send free-form messages only within 24 hours of the user's last message.